

```

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#define MAX_PRODUCTS 500

#define MAX_NAME 50

#define MIN_STOCK 20 //

// Estructura que define el modelo del producto

typedef struct {

    char codigo[15];

    char nombre[MAX_NAME];

    int cantidad;

    char lote[15];

    double precio;

    char tipo_botella[15]; // [CUMPLE REQ002] Formatos: BOT600, BOT1LT, BOT5LT, BIDON

} Producto;

// Base de datos temporal en memoria

Producto inventario[MAX_PRODUCTS];

int total_productos = 0;

// CUMPLE REQ008 Almacenamiento histórico de transacciones

int total_entradas = 0;

int total_salidas = 0;

// Prototipos de funciones

void limpiarBuffer();

void menuPrincipal();

void registrarProducto();

void consultarInventario();

void buscarProducto();

void actualizarStock();

void eliminarProducto();

void verificarStockBajo();

void generarInformeDiario();

void limpiarBuffer() {

    int c;

    while ((c = getchar()) != '\n' && c != EOF);

}

// REQ001 Función Main que gestiona el flujo centralizado del menú

int main() {

    int opcion;

    do {

```

```

printf("\n=====\\n");

printf("    SISTEMA DE INVENTARIO - VITALIC    \\n");

printf("=====\\n");

printf("1. Registrar nuevo producto  (REQ002)\\n");

printf("2. Consultar inventario      (REQ003)\\n");

printf("3. Buscar un producto          (REQ004)\\n");

printf("4. Actualizar stock            (REQ005)\\n");

printf("5. Eliminar producto           (REQ006)\\n");

printf("6. Alerta de stock bajo        (REQ007)\\n");

printf("7. Generar informe diario      (REQ008)\\n");

printf("8. Salir del programa\\n");

printf("=====\\n");

printf("Seleccione una opción: ");

if (scanf("%d", &opcion) != 1) {

    printf("[ERROR] Por favor, ingrese una opción numérica válida.\\n");

    limpiarBuffer();

    opcion = 0;

    continue;

}

limpiarBuffer();

switch (opcion) {

    case 1:

        registrarProducto(); // Ejecuta REQ002

        break;

    case 2:

        consultarInventario(); // Ejecuta REQ003

        break;

    case 3:

        buscarProducto(); // Ejecuta REQ004

        break;

    case 4:

        actualizarStock(); // Ejecuta REQ005

        break;

    case 5:

        eliminarProducto(); // Ejecuta REQ006

        break;

    case 6:

        verificarStockBajo(); // Ejecuta REQ007

```

```

        break;

    case 7:

        generarInformeDiario(); // Ejecuta REQ008

        break;

    case 8:

        printf("\nSaliendo del programa de inventario de botellas. ¡Hasta pronto!\n");

        break;

    default:

        printf("[ERROR] Opción no válida. Intente nuevamente.\n");

        break;

}

} while (opcion != 8);

return 0;

}

```

#### // REQ002 Registro de nuevos productos en el inventario

```

void registrarProducto() {

    if (total_productos >= MAX_PRODUCTS) {

        printf("\n[ERROR] El inventario está lleno. Límite de capacidad alcanzado.\n");

        return;

    }

    Producto p;

    printf("\n=== REGISTRO DE PRODUCTO (REQ002) ===\n");

    printf("Ingrese el código único del producto: ");

    scanf("%14s", p.codigo);

    limpiarBuffer();

    // Validar que el código no exista previamente en el almacén

    for (int i = 0; i < total_productos; i++) {

        if (strcmp(inventario[i].codigo, p.codigo) == 0) {

            printf("[ERROR] Ya existe un producto registrado con el código '%s'.\n", p.codigo);

            return;

        }

    }

    printf("Ingrese el nombre del producto: ");

    fgets(p.nombre, MAX_NAME, stdin);

    p.nombre[strcspn(p.nombre, "\n")] = 0; // Quitar el salto de línea sobrante

    printf("Ingrese la cantidad inicial: ");

    while (scanf("%d", &p.cantidad) != 1 || p.cantidad < 0) {

        printf("[ERROR] Cantidad incorrecta. Ingrese un entero positivo: ");

        limpiarBuffer();

    }

}

```

```

    }

    limpiarBuffer();

    printf("Ingrese el lote del producto: ");

    scanf("%14s", p.lote);

    limpiarBuffer();

    printf("Ingrese el precio unitario: ");

    while (scanf("%lf", &p.precio) != 1 || p.precio < 0.0) {

        printf("[ERROR] Precio incorrecto. Ingrese un valor numérico: ");

        limpiarBuffer();

    }

    limpiarBuffer();

    // Selección de presentación de botella solicitada en REQ002

    int opcion_botella;

    printf("\nSeleccione la presentación de la botella:\n");

    printf("1. 600ml (BOT600)\n");

    printf("2. 1 Litro (BOT1LT)\n");

    printf("3. 5 Litros (BOT5LT)\n");

    printf("4. Bidón (BIDON)\n");

    printf("Opción: ");

    while (scanf("%d", &opcion_botella) != 1 || opcion_botella < 1 || opcion_botella > 4) {

        printf("[ERROR] Opción no válida. Ingrese de 1 a 4: ");

        limpiarBuffer();

    }

    limpiarBuffer();

    switch (opcion_botella) {

        case 1: strcpy(p.tipo_botella, "BOT600"); break;

        case 2: strcpy(p.tipo_botella, "BOT1LT"); break;

        case 3: strcpy(p.tipo_botella, "BOT5LT"); break;

        case 4: strcpy(p.tipo_botella, "BIDON"); break;

    }

    // Agregar al arreglo

    inventario[total_productos] = p;

    total_productos++;

    // Guardar para informe diario (REQ008)

    total_entradas += p.cantidad;

    printf("\n[ÉXITO] Producto registrado exitosamente bajo la clasificación: %s\n", p.tipo_botella);

}

```

// REQ003 Consulta global de inventario en formato de columnas tabulado

```

void consultarInventario() {

```

```

printf("\n=== CONSULTA DE INVENTARIO (REQ003) ===\n");

if (total_productos == 0) {

    printf("[INFO] No hay productos en el inventario actual.\n");

    return;

}

printf("-----\n");

printf("%-12s | %-20s | %-12s | %-10s | %-10s | %-10s\n", "CÓDIGO", "NOMBRE", "PRESENTACIÓN", "CANTIDAD", "LOTE", "PRECIO");

printf("-----\n");

for (int i = 0; i < total_productos; i++) {

    printf("%-12s | %-20s | %-12s | %-10d | %-10s | $%-9.2f\n",

        inventario[i].codigo,

        inventario[i].nombre,

        inventario[i].tipo_botella,

        inventario[i].cantidad,

        inventario[i].lote,

        inventario[i].precio);

}

printf("-----\n");
}

```

#### // REQ004 Búsqueda inteligente de productos por código o subcadena de nombre

```

void buscarProducto() {

    printf("\n=== BÚSQUEDA DE PRODUCTOS (REQ004) ===\n");

    if (total_productos == 0) {

        printf("[INFO] El inventario está vacío.\n");

        return;

    }

    char criterio[MAX_NAME];

    printf("Ingrese el nombre o código a buscar: ");

    fgets(criterio, MAX_NAME, stdin);

    criterio[strcspn(criterio, "\n")] = 0; // Quitar el salto de línea

    int encontrado = 0;

    for (int i = 0; i < total_productos; i++) {

        // Coincidencia exacta con código o parcial/completa con el nombre del producto

        if (strcmp(inventario[i].codigo, criterio) == 0 || strstr(inventario[i].nombre, criterio) != NULL) {

            if (!encontrado) {

                printf("\nProducto(s) localizado(s):\n");

                printf("-----\n");

                printf("%-12s | %-20s | %-12s | %-10s | %-10s | %-10s\n", "CÓDIGO", "NOMBRE", "PRESENTACIÓN", "CANTIDAD", "LOTE",

"PRECIO");

```

```

        printf("-----\n");
    }

    printf("%-12s | %-20s | %-12s | %-10d | %-10s | $%-9.2f\n",
        inventario[i].codigo,
        inventario[i].nombre,
        inventario[i].tipo_botella,
        inventario[i].cantidad,
        inventario[i].lote,
        inventario[i].precio);

    encontrado = 1;
}
}

if (encontrado) {
    printf("-----\n");
} else {
    printf("[Aviso] No se encontró ningún producto bajo el criterio '%s'.\n", criterio);
}
}

```

#### // REQ005 Actualización y modificación de las cantidades de inventario

```

void actualizarStock() {
    printf("\n=== ACTUALIZACIÓN DE STOCK (REQ005) ===\n");
    if (total_productos == 0) {
        printf("[INFO] El inventario está vacío.\n");
        return;
    }

    char codigo[15];
    printf("Ingrese el código del producto a modificar: ");
    scanf("%14s", codigo);
    limpiarBuffer();
    int indice = -1;
    for (int i = 0; i < total_productos; i++) {
        if (strcmp(inventario[i].codigo, codigo) == 0) {
            indice = i;
            break;
        }
    }

    if (indice == -1) {
        printf("[ERROR] No se encontró ningún producto con ese código.\n");
        return;
    }
}

```

```

    }

    printf("Producto seleccionado: %s (Cantidad actual en almacén: %d)\n", inventario[indice].nombre, inventario[indice].cantidad);

    int nueva_cantidad;

    printf("Ingrese la nueva cantidad: ");

    while (scanf("%d", &nueva_cantidad) != 1 || nueva_cantidad < 0) {

        printf("[ERROR] Cantidad incorrecta. Ingrese un número positivo: ");

        limpiarBuffer();

    }

    limpiarBuffer();

    // Calcular flujos para balance del informe diario (REQ008)

    int diferencia = nueva_cantidad - inventario[indice].cantidad;

    if (diferencia > 0) {

        total_entradas += diferencia; // Aumentó el stock

    } else if (diferencia < 0) {

        total_salidas += (-diferencia); // Disminuyó el stock (salida de almacén)

    }

    inventario[indice].cantidad = nueva_cantidad;

    printf("ÉXITO] Stock del producto '%s' modificado a %d unidades.\n", inventario[indice].nombre, nueva_cantidad);

}

// REQ006 Retirar/Eliminar definitivamente un producto descontinuado

void eliminarProducto() {

    printf("\n=== ELIMINAR PRODUCTO DEL INVENTARIO (REQ006) ===\n");

    if (total_productos == 0) {

        printf("[INFO] El inventario está vacío.\n");

        return;

    }

    char codigo[15];

    printf("Ingrese el código del producto a dar de baja: ");

    scanf("%14s", codigo);

    limpiarBuffer();

    int indice = -1;

    for (int i = 0; i < total_productos; i++) {

        if (strcmp(inventario[i].codigo, codigo) == 0) {

            indice = i;

            break;

        }

    }

}

if (indice == -1) {

```

```

        printf("[ERROR] No se encontró ningún producto con ese código.\n");

        return;
    }

    char confirmacion;

    printf("¿Está completamente seguro de eliminar el producto '%s'? (S/N): ", inventario[indice].nombre);

    scanf("%c", &confirmacion);

    limpiarBuffer();

    if (confirmacion == 'S' || confirmacion == 's') {

        // REQ008 El stock que se elimina se contabiliza como salida total del producto

        total_salidas += inventario[indice].cantidad;

        // Desplazamiento de los elementos para borrar del arreglo

        for (int i = indice; i < total_productos - 1; i++) {

            inventario[i] = inventario[i + 1];

        }

        total_productos--;

        printf("[ÉXITO] Producto eliminado de forma definitiva del almacén.\n");

    } else {

        printf("[INFO] Operación abortada. El producto no fue eliminado.\n");

    }

}

//REQ007 Alerta que identifica si un producto está por debajo del límite mínimo

void verificarStockBajo() {

    printf("\n=== ALERTAS DE STOCK BAJO (REQ007) ===\n");

    int alertas_disparadas = 0;

    for (int i = 0; i < total_productos; i++) {

        if (inventario[i].cantidad < MIN_STOCK) {

            if (alertas_disparadas == 0) {

                printf("¡ATENCIÓN! Los siguientes productos tienen inventario crítico (< %d unidades):\n", MIN_STOCK);

                printf("-----\n");

                printf("%-12s | %-20s | %-12s | %-10s\n", "CÓDIGO", "NOMBRE", "PRESENTACIÓN", "STOCK ACTUAL");

                printf("-----\n");

            }

            printf("%-12s | %-20s | %-12s | %-10d (BAJO STOCK)\n",

                inventario[i].codigo,

                inventario[i].nombre,

                inventario[i].tipo_botella,

                inventario[i].cantidad);

            alertas_disparadas++;

        }

    }
}

```



```

    }

    if (alertas_disparadas > 0) {

        printf("-----\n");

    } else {

        printf("[INFO] Todo el stock se encuentra en rangos seguros (Ninguno por debajo de %d unidades).\n", MIN_STOCK);

    }

}

```

**// REQ008 Generar, mostrar en pantalla y guardar el informe diario en archivo físico (.txt)**

```

void generarInformeDiario() {

    printf("\n=== INFORME DIARIO DE INVENTARIO (REQ008) ===\n");

    int stock_actual_total = 0;

    double valor_total_inventario = 0.0;

    for (int i = 0; i < total_productos; i++) {

        stock_actual_total += inventario[i].cantidad;

        valor_total_inventario += (inventario[i].cantidad * inventario[i].precio);

    }

    // Mostrar el informe en pantalla

    printf("\n--- Resumen del Estado de Inventario ---\n");

    printf("Total de productos distintos: %d\n", total_productos);

    printf("Unidades físicas actuales en stock: %d\n", stock_actual_total);

    printf("Entradas de producto del día: %d unidades\n", total_entradas);

    printf("Salidas de producto del día: %d unidades\n", total_salidas);

    printf("Valor financiero del inventario actual: $%.2f\n", valor_total_inventario);

    // Guardar el informe en el archivo local 'informe_diario.txt'

    FILE *archivo = fopen("informe_diario.txt", "w");

    if (archivo == NULL) {

        printf("[ERROR] No se pudo guardar el informe en un archivo local.\n");

        return;

    }

    fprintf(archivo, "=====\n");

    fprintf(archivo, "    REPORTE DIARIO DE INVENTARIO    \n");

    fprintf(archivo, "=====\n");

    fprintf(archivo, "Total de Productos Registrados: %d\n", total_productos);

    fprintf(archivo, "Unidades en Stock Físico actual: %d\n", stock_actual_total);

    fprintf(archivo, "Histórico acumulado Entradas del Día: %d\n", total_entradas);

    fprintf(archivo, "Histórico acumulado Salidas del Día: %d\n", total_salidas);

    fprintf(archivo, "Valorización del inventario actual: $%.2f\n", valor_total_inventario);

    fprintf(archivo, "=====\n");

    fprintf(archivo, "Detalle Técnico de Existencias:\n");

```

```
fprintf(archivo, "%-12s | %-20s | %-12s | %-10s | %-10s\n", "CÓDIGO", "NOMBRE", "PRECIO", "LOTE", "CATEGORÍA");  
  
for (int i = 0; i < total_productos; i++) {  
  
    fprintf(archivo, "%-12s | %-20s | $%-9.2f | %-10s | %-12s\n",  
  
        inventario[i].codigo,  
  
        inventario[i].nombre,  
  
        inventario[i].precio,  
  
        inventario[i].lote,  
  
        inventario[i].tipo_botella);  
  
}  
  
fclose(archivo);  
  
printf("\n[ÉXITO] Informe guardado localmente con el nombre 'informe_diario.txt'.\n");  
  
}
```